



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

KHOA TOÁN – TIN

Lập trình Hướng Đối tượng Object Oriented Programming

\$11 Tập hợp





Nội dung

- Collection
- Các lớp đối tượng cài đặt: List, Set, Queue, Dictionary, ...





ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

KHOA TOÁN – TIN

1. Collection



Collection

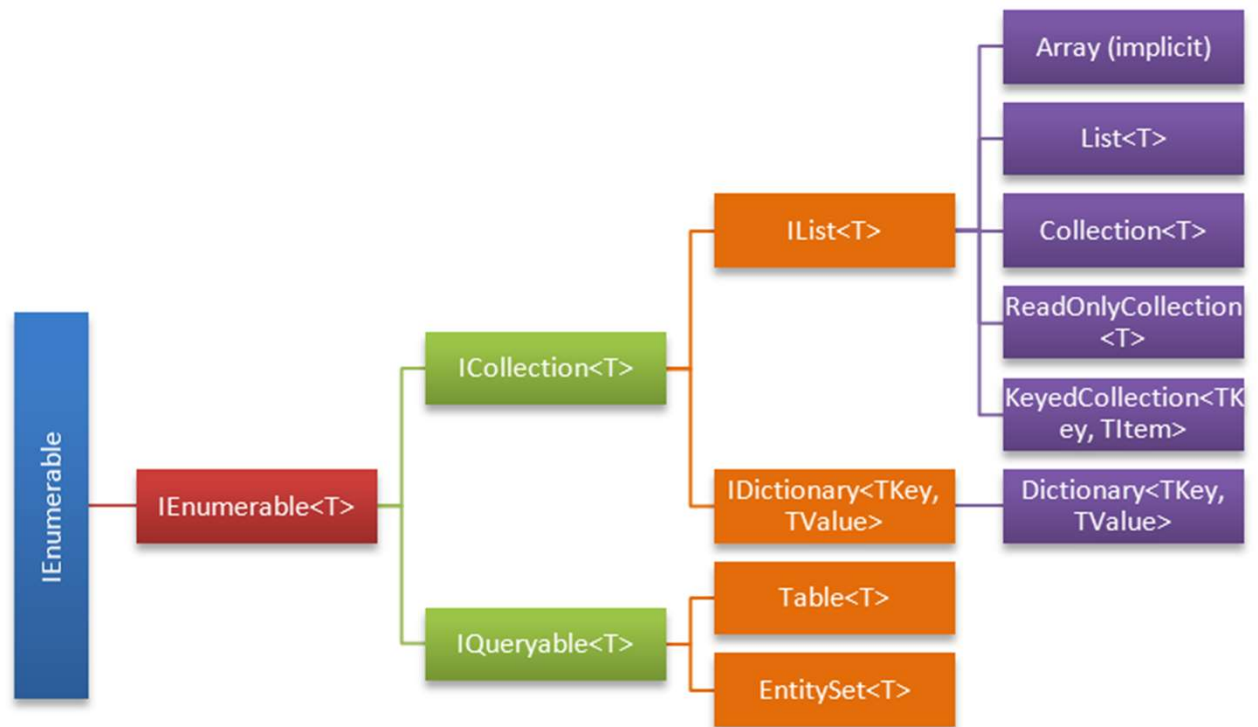
- Thao tác với tập hợp nhiều đối tượng - tác vụ phổ biến
- Tập hợp trong Lập trình thủ tục
 - Array
 - List
- Hướng đối tượng
 - Collection cung cấp cơ chế làm việc với nhiều loại tập hợp
 - Các tác vụ cơ bản:
 - thêm/bớt phần tử
 - duyệt, tìm kiếm
 - sắp xếp, ...

Collection .Net

- .Net cung cấp phương tiện làm việc với Collection
 - namespace: `System.Collection` ; `System.Collection.Generic` ; ...
 - Có nhiều interface liên quan

Interface	Mô tả
<u>IEnumerable<T></u>	Triển khai nó nếu muốn duyệt phần tử bằng foreach, nó định nghĩa phương thức GetEnumerator trả về một enumerator.
<u>ICollection<T></u>	Giao diện này được triển khai bởi các generic collection. Với nó lấy tổng phần tử bằng thuộc tính Count, copy các phần tử vào mảng bằng CopyTo, thêm bớt phần tử với Add, Remove, Clear
<u>IList<T></u>	Giao diện này kế thừa ICollection<T> là một danh sách các phần tử truy cập được theo vị trí của nó. Nó có indexer, phương thức để chèn phần tử xóa phần tử Insert RemoveAt.
<u>ISet<T></u>	Giao diện triển khai bởi các tập hợp
<u>IDictionary<TKey,TValue></u>	Giao diện để triển khai loại dữ liệu lưu trữ theo cặp key, value.
<u>ILookup<TKey,TValue></u>	Giao diện để triển khai loại dữ liệu lưu trữ theo cặp key, value. Nhưng cho phép một key có nhiều giá trị
<u>IComparer<TKey,TValue></u>	Giao diện để triển khai cho phép so sánh để sắp xếp Collection
<u>IEqualityComparer<TKey,TValue></u>	Giao diện để triển khai cho phép so sánh bằng

Collection .Net





ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

KHOA TOÁN – TIN

Các lớp đối tượng Collection



List

- Là lớp triển khai các giao diện IList, ICollection, IEnumerable
- Chứa tập hợp các đối tượng cùng kiểu
- Các method chính:
 - Add, AddRange, Insert, RemoveAt, Remove, ...

List

- Khởi tạo:

- `List<type> = new List<type>`

```
public class Product {  
    public int ID {set; get;}  
    public string Name {set; get;}  
    public double Price {set; get;}  
}  
var numbers = new List<int>();  
List<Product> products = new List<Product>();
```

List

- Thêm - Add, xóa - RemoveAt, ..., chèn phần tử - Insert

```
var p = new Product(2, "iPhone 7", 200);  
products.Add(p); //thêm vào cuối  
var arrayProducts = new Product[2]; ...  
products.AddRange(arrayProducts); //nối phần tử của array vào list  
products.Insert(3, new Product(6, "Macbook Pro", 1000)); // chèn phần tử vào vị trí index 3, (thứ 4)  
products.RemoveAt(0); // xóa phần tử đầu tiên
```

- Duyệt: for, foreach

```
for (int i = 1; i < products.Count; i++) {  
}  
// Duyệt qua các phần tử bằng foreach  
foreach (var pi in products) {  
    Console.WriteLine(pi.ToString());  
}
```

List

■ Tìm kiếm, sắp xếp

<code>IndexOf(obj)</code>	Tìm index của đối tượng trong List
<code>LastIndexOf(obj)</code>	Tìm index của phần tử cuối cùng có giá trị bằng obj trong List
<code>FindIndex</code>	Tìm kiếm trả về Index
<code>FindLastIndex</code>	Tìm kiếm trả về Index cuối
<code>Find(Predicate)</code>	Tìm kiếm trả về phần tử
<code>FindAll(Predicate)</code>	Tìm kiếm trả về danh sách phần tử
<code>FindLast</code>	Tìm kiếm trả về phần tử cuối tìm thấy
<code>Sort</code>	Sắp xếp (triển khai <code>IComparable</code> - method <code>CompareTo</code>)

Hashtable

- Lưu tập dữ liệu dạng {key, value}
- Truy xuất phần tử qua key
- Tìm kiếm theo cơ chế băm (hash) - nhanh

Hashtable

■ Một số Phương thức

Count	Số lượng phần tử hiện có trong Hashtable.
Keys	Danh sách các key của phần tử trong Hashtable.
Values	Danh sách các value của phần tử trong Hashtable.
Add(Object, Object)	Thêm một phần tử với cặp key-value vào Hashtable.
Remove(Object)	Xóa một phần tử có key xuất hiện đầu tiên.
Clone()	Tạo một bản sao từ Hashtable
ContainsKey(Object)	Kiểm tra key có tồn tại trong danh sách key của Hashtable hay không.
ContainsValue(Object)	Kiểm tra value có tồn tại trong danh sách value của Hashtable hay không.

Hashtable

```
Hashtable openWith = new Hashtable();

// Add some elements to the hash table. There are no
// duplicate keys, but some of the values are duplicates.
openWith.Add("txt", "notepad.exe");
openWith.Add("bmp", "paint.exe");

// ContainsKey can be used to test keys before inserting
// them.
if (!openWith.ContainsKey("ht"))
{
    openWith.Add("ht", "hypertrm.exe");
    Console.WriteLine("Value added for key = \"ht\": {0}", openWith["ht"]);
}
```

SortedList

- Một collection
 - Lưu trữ dạng {key-value}
 - Phần tử được sắp xếp theo key
 - Kết hợp Hashtable và ArrayList
 - Truy xuất phần tử: index hoặc key

SortedList

■ Các method chính

Count	Trả về số phần tử hiện có trong SortedList.
Keys	Trả về danh sách key trong SortedList.
Values	Trả về danh sách value trong SortedList.
Add(Object, Object)	Thêm một phần tử với key-value vào SortedList.
GetByIndex(Int32)	Trả về giá trị phần tử tại vị trí index.
SetByIndex(Int32, Object)	Thay thế giá trị cho phần tử tại vị trí index.
ContainsKey(Object)	Kiểm tra key có tồn tại trong SortedList hay không.
ContainsValue(Object)	Kiểm tra value có tồn tại trong SortedList hay không.
Clone()	Tạo ra một bản sao của SortedList.

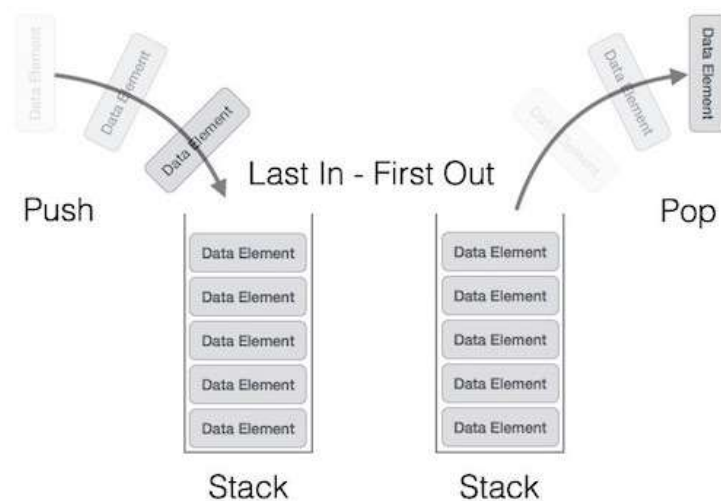
SortedList

```
// Creates and initializes a new SortedList.
SortedList mySL = new SortedList();
mySL.Add("Third", "!");
mySL.Add("Second", "World");
mySL.Add("First", "Hello");
// Displays the properties and values of the SortedList.
Console.WriteLine("  Count:    {0}", mySL.Count);
Console.WriteLine("  Capacity: {0}", mySL.Capacity);
// Search by key
int indx = mySL.IndexOfKey("First");
Console.WriteLine($"  Values: {mySL.GetByIndex(indx)}");
PrintKeysAndValues(mySL);

public static void PrintKeysAndValues(SortedList myList)
{
    for (int i = 0; i < myList.Count; i++)
    {
        Console.WriteLine("\t{0}:\t{1}", myList.GetKey(i), myList.GetByIndex(i));
    }
}
```

Stack - ngăn xếp

- Nguyên tắc LIFO



- Một số method

Count	Số phần tử hiện có trong stack.
Push(Object)	Thêm một phần tử vào trí trên cùng của stack.
Pop()	trả về phần tử được thêm vào cuối cùng và xóa phần tử đó khỏi stack.
Peek()	trả về phần tử được thêm vào cuối cùng nhưng không xóa phần tử đó khỏi stack.
Contains(Object)	Kiểm tra đối tượng có tồn tại trong stack không.

Stack - ngăn xếp

Count	Số phần tử hiện có trong stack.
Push(Object)	Thêm một phần tử vào trí trên cùng của stack.
Pop()	trả về phần tử được thêm vào cuối cùng và xóa phần tử đó khỏi stack.
Peek()	trả về phần tử được thêm vào cuối cùng nhưng không xóa phần tử đó khỏi stack.
Contains(Object)	Kiểm tra đối tượng có tồn tại trong stack không.

```
Stack myStack = new Stack();  
myStack.Push("Hello");  
myStack.Push("World");  
myStack.Push("!");  
myStack.Peek();  
myStack.Contains("Hello");
```

Queue - Hàng đợi

- Nguyên tắc FIFO

- Một số method

Count	Số phần tử hiện có trong Queue.
Enqueue(Object)	Thêm một phần tử vào cuối Queue.
Dequeue()	trả về phần tử đầu tiên và xóa phần tử đó khỏi Queue.
Peek()	trả về phần tử đầu tiên nhưng không xóa phần tử đó khỏi Queue.
Contains(Object)	Kiểm tra đối tượng có tồn tại trong Queue không.

Queue - Hàng đợi

Count	Số phần tử hiện có trong Queue.
Enqueue(Object)	Thêm một phần tử vào cuối Queue.
Dequeue()	trả về phần tử đầu tiên và xóa phần tử đó khỏi Queue.
Peek()	trả về phần tử đầu tiên nhưng không xóa phần tử đó khỏi Queue.
Contains(Object)	Kiểm tra đối tượng có tồn tại trong Queue không.

```
// Creates and initializes a new Queue.  
Queue myQ = new Queue();  
  
myQ.Enqueue("Hello");  
myQ.Enqueue("World");  
myQ.Enqueue("!");  
Console.WriteLine( "\tCount:    {0}", myQ.Count );  
var item = myQ.Dequeue();
```

ArrayList

- Đối tượng collection
- Lưu trữ mảng phần tử (các phần tử có thể kiểu khác nhau: khác List)
- Các method

Count	Trả về số phần tử hiện có của ArrayList
Add(Object)	Thêm một phần tử vào cuối ArrayList
AddRange(ICollection)	Thêm danh sách phần tử vào cuối ArrayList
Clone()	tạo bản sao cho ArrayList
Insert(int Index, object Value)	Thêm một phần tử vào vị trí index
Remove(object Value)	Xóa phần tử đầu tiên xuất hiện trong ArrayList

ArrayList

Count	Trả về số phần tử hiện có của ArrayList
Add(Object)	Thêm một phần tử vào cuối ArrayList
AddRange(ICollection)	Thêm danh sách phần tử vào cuối ArrayList
Clone()	tạo bản sao cho ArrayList
Insert(int Index, object Value)	Thêm một phần tử vào vị trí index
Remove(object Value)	Xóa phần tử đầu tiên xuất hiện trong ArrayList

```
ArrayList myAL = new ArrayList();  
myAL.Add("Hello");  
myAL.Add("World");  
myAL.Add("!");  
myAL.Add(100);
```

Dictionary

- Là một đối tượng tập hợp (System.Collection.Generic) quản lý dữ liệu dạng {key-value},
 - Key và Value có thể là kiểu dữ liệu bất kỳ
 - Key không được trùng
- Tốc độ tìm kiếm, thêm, xóa phần tử nhanh
- Khác hashtable:
 - Hashtable là tập hợp đối tượng cụ thể
 - Dictionary là tập hợp đối tượng chung tổng quát

Dictionary<TKey, TValue>

Dictionary

- Một số prop và method:

Count	Thuộc tính cho biết số phần tử
[key]	Indexer truy cập đến phần tử có key
Keys	Thuộc tính là danh sách các key
Values	Thuộc tính lấy danh sách các giá trị
Add(key, value)	Thêm một phần tử vào Dictionary
Remove(key)	Xóa phần tử bằng key của nó
Clear()	Loại bỏ tất cả các phần tử
ContainKey(key)	Kiểm tra có phần tử nào có khóa là key
ContainValue(value)	Kiểm tra có phần tử nào có giá trị là value

Dictionary

- Key và Value có thể là kiểu dữ liệu bất kỳ

```
public class Department
{
    public string Name { get; set; }
    public string Address { get; set; }
    Department(string name, string address)
    {
        Name = name;
        Address = address;
    }
}
```

```
Dictionary<string, Department> departmentList = new Dictionary<string, Department>();
departmentList.Add("dep01", new Department("Engineering", "Room 203"));
departmentList.Add("dep02", new Department("Planing", "Room 403"));
```

Dictionary

```
Dictionary<string, string> openWith = new Dictionary<string, string>();

// Add some elements to the dictionary. There are no
// duplicate keys, but some of the values are duplicates.
openWith.Add("txt", "notepad.exe");
openWith.Add("bmp", "paint.exe");
openWith.Add("dib", "paint.exe");

// The Add method throws an exception if the new key is already in the dictionary.
try
{
    openWith.Add("txt", "winword.exe");
}
catch (ArgumentException)
{
    Console.WriteLine("An element with Key = \"txt\" already exists.");
}
```

Dictionary

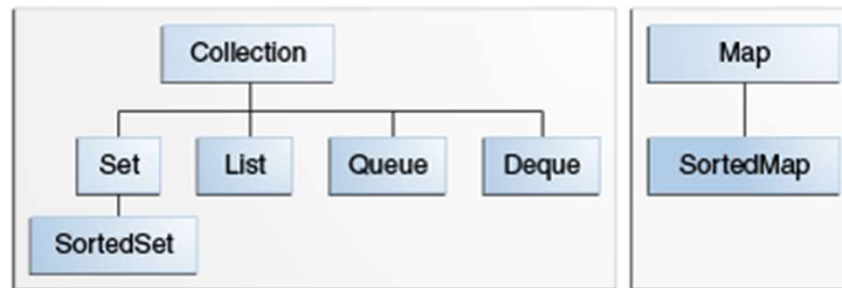
- duyệt: lệnh foreach

```
// When you use foreach to enumerate dictionary elements,  
// the elements are retrieved as KeyValuePair objects.  
Console.WriteLine();  
foreach( KeyValuePair<string, string> kvp in openWith )  
{  
    Console.WriteLine("Key = {0}, Value = {1}",  
        kvp.Key, kvp.Value);  
}
```

Dictionary

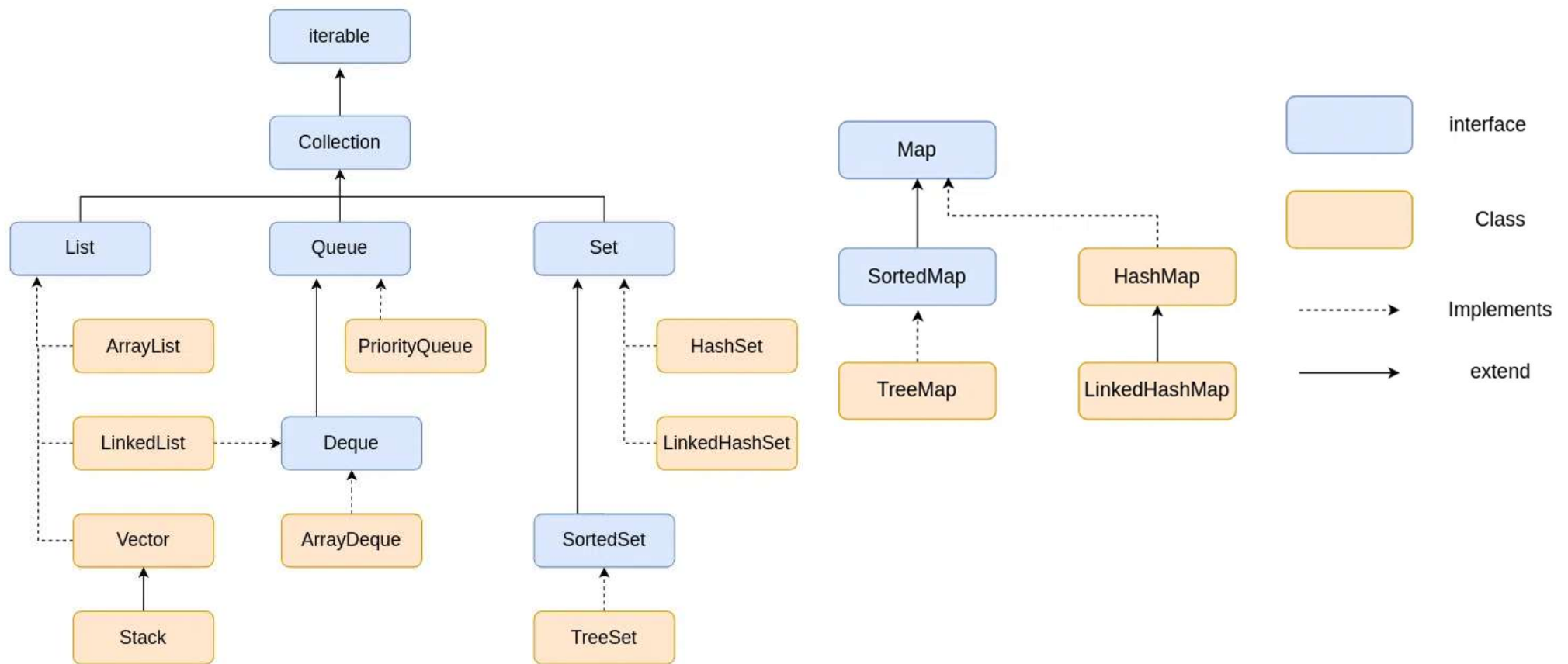
```
// When a program often has to try keys that turn out not to be in the dictionary,  
// TryGetValue can be a more efficient way to retrieve values.  
string value = "";  
if (openWith.TryGetValue("tif", out value)) {  
    Console.WriteLine("For key = \"tif\", value = {0}.", value);  
}  
else {  
    Console.WriteLine("Key = \"tif\" is not found.");  
}  
  
// ContainsKey can be used to test keys before inserting them.  
if (!openWith.ContainsKey("ht"))  
{  
    openWith.Add("ht", "hypertrm.exe");  
    Console.WriteLine("Value added for key = \"ht\": {0}",  
        openWith["ht"]);  
}
```

Collection trong java



- Collection Interface định nghĩa những phương thức cơ bản khi làm việc với tập hợp
- Map là kiểu cho phép quản lý dữ liệu theo dạng {key-value}, trong đó key là duy nhất và tương ứng với 1 key là một giá trị value

Collection trong java





Collection trong python

- Cung cấp khả năng lưu, thao tác với tập hợp phần tử dữ liệu
- Được giới thiệu từ python 2.4
- Module collection



Collection trong python

- Các kiểu dữ liệu tập hợp cơ bản:
 - List: collection có thứ tự, có thể thay đổi
 - Tuple: collection có thứ tự, không thể thay đổi
 - Set: collection không có thứ tự, không có chỉ mục, kh trùng lặp
 - Dictionary: collection không có thứ tự, có chỉ mục, kh trùng lặp

Collection trong python

■ VD

```
list1 = [1, 2, 3, 4, [5, 6, 7], "A", 'B']  
print(list1[3]) # --> 4  
list1[3] = 10  
print(len(list1)) # --> 7  
list1.append("C") #thêm phần tử  
list1.remove('B') #xóa phần tử
```

```
tuple1 = (1, 2, 3, 4, (5, 6, 7), "A", 'B')  
tuple1[3] = 10 # --> TypeError: //tuple' object does not support item assignment  
print (tuple1.index('A')) #--> 5
```

```
set1 = {4, 3, 5, 2, 1, 6, 2}  
set1.add(100)
```

Collection trong python

- VD

```
dictCar = {  
    "brand": "Honda",  
    "model": "Honda Civic",  
    "year": 1972  
}  
print(dictCar.keys()) # --> dict_keys(['brand', 'model', 'year'])  
print(dictCar.items())  
#truy cập qua key  
print(dictCar['brand']) # --> Honda
```

Bài tập

- In a method, declare a variable temperatures of type List.
- Add some numbers to the list.
- Write a foreach loop to count the number of temperatures that equal or exceed 25 degrees.
- Write a method GreaterCount with signature static int GreaterCount(List list, double min) { ... } that returns the number of elements of list that are greater than or equal to min. Call the method on your temperatures list.